

Spring Data JPA

JpaRepository • Query by Example • Batch Operations • Derived Query Methods

This document combines two topics: the advanced features of **JpaRepository** (getReferenceById, Query by Example, batch deletes) and Spring Data JPA's powerful **Derived Query Methods** — where SQL is generated automatically from method names.

JpaRepository, Query by Example & Batch Operations

1. Repository Upgrade — JpaRepository

```
package com.telusko.repo;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.telusko.entity.Product;

@Repository
public interface IProductRepo extends JpaRepository<Product, Integer>
{
}
```

You have now moved to the top of the repository inheritance chain:

```
CrudRepository          <- 11 basic CRUD methods (save, findById, delete...)
|
PagingAndSortingRepository <- adds findAll(Sort) and findAll(Pageable)
|
JpaRepository           <- adds getReferenceById, deleteAllByIdInBatch,
                           findAll(Example), flush(), saveAndFlush()...
```

Extending **JpaRepository<Product, Integer>** gives you every method from all three levels automatically — no extra configuration needed.

2. Service Interface — IProductService.java

```
package com.telusko.service;

import java.util.List;
import com.telusko.entity.Product;

public interface IProductService
{
    Product searchProductById(Integer id);
    List<Product> searchProductsByIds(Iterable<Integer> ids);
    List<Product> searchProductsByProduct(Product product);
    String deleteProductsByIdsCrud(List<Integer> ids);
    String deleteProductsByIds(List<Integer> ids);
}
```

3. Service Implementation — ProductService.java

```
package com.telusko.service;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Example;
import org.springframework.stereotype.Service;
import com.telusko.entity.Product;
import com.telusko.repo.IProductRepo;

@Service
public class ProductService implements IProductService
{
    @Autowired
    private IProductRepo repo;

    @Override
    public Product searchProductById(Integer id)
    {
        return repo.getReferenceById(id);
    }

    @Override
    public List<Product> searchProductsByIds(Iterable<Integer> ids)
    {
        return repo.findAllById(ids);
    }

    @Override
    public List<Product> searchProductsByProduct(Product product)
    {
        Example<Product> example = Example.of(product);
        return repo.findAll(example);
    }

    @Override
    public String deleteProductsByIdsCrud(List<Integer> ids)
    {
        List<Product> products = (List<Product>) repo.findAllById(ids);
        if (products.size() != 0)
        {
            repo.deleteAllById(ids);
            return "All Products records with given ids deleted";
        }
        else
        {

```

```

        return "Product info with given ids are not in records to delete";
    }
}

@Override
public String deleteProductsByIds(List<Integer> ids)
{
    List<Product> products = (List<Product>) repo.findAllById(ids);
    if (products.size() != 0)
    {
        repo.deleteAllByIdInBatch(ids);
        return "All Products records with given ids deleted";
    }
    else
    {
        return "Product info with given ids are not in records to delete";
    }
}
}

```

4. Key Concepts Explained

4.1 getReferenceById vs findById

Method	DB Hit Timing	Returns	If Not Found
findById(id)	Immediately	Optional<T>	Empty Optional
getReferenceById(id)	Lazy – on field access	Proxy object	EntityNotFoundException

getReferenceById returns a Hibernate proxy. The database is only queried when you actually read a field on the returned object. This is useful when you only need a reference (e.g. to set as a foreign key) without loading the full row.

4.2 Query by Example (QBE)

Query by Example lets you search using an actual entity object as the search template — no SQL, no method name derivation, no @Query annotation needed.

```

// Usage from main:
service.searchProductsByProduct(
    new Product("Mobile", "Electronics", 5000.0, 20)
);

// Inside service:
Example<Product> example = Example.of(product);

```

```
return repo.findAll(example);
```

Spring inspects the non-null fields of the object and builds a WHERE clause automatically:

```
SELECT * FROM product
WHERE product_name = 'Mobile'
  AND category     = 'Electronics'
  AND price        = 5000.0
  AND quantity     = 20
```

■ Null fields are automatically ignored. Pass new Product(null, "Electronics", null, null) and it generates: WHERE category = 'Electronics' — great for dynamic search filters.

4.3 deleteAllById vs deleteAllByIdInBatch

Both methods look similar but generate very different SQL:

deleteProductsByIdsCrud — one DELETE per id

```
repo.deleteAllById(ids);

// SQL generated (3 separate round-trips):
DELETE FROM product WHERE id = 3;
DELETE FROM product WHERE id = 5;
DELETE FROM product WHERE id = 52;
```

deleteProductsByIds — single batch DELETE

```
repo.deleteAllByIdInBatch(ids);

// SQL generated (1 round-trip):
DELETE FROM product WHERE id IN (3, 5, 52);
```

Method	SQL Generated	DB Round-trips	Use When
deleteAllById(ids)	One DELETE per id	N trips	Small lists, need lifecycle hooks
deleteAllByIdInBatch(ids)	Single DELETE IN (...)	1 trip	Large lists, performance matters

5. Main Application

```
@SpringBootApplication
public class Datajpa1crudApplication {

    public static void main(String[] args) {
        ConfigurableApplicationContext container =
            SpringApplication.run(Datajpa1crudApplication.class, args);
        IProductService service = container.getBean(IProductService.class);

        // Search by single id
```

```

//      System.out.println(service.searchProductById(52));

//      Search by multiple ids
//      List<Integer> ids = Arrays.asList(53, 54, 55);
//      service.searchProductsByIds(ids).forEach(p -> System.out.println(p));

//      Query by Example
//      System.out.println(
//          service.searchProductsByProduct(
//              new Product("Mobile", "Electronics", 5000.0, 20)));

//      Batch delete (active)
//      List<Integer> ids = Arrays.asList(3, 5, 52);
//      String status = service.deleteProductsByIds(ids);
//      System.out.println(status);
    }
}

```

6. Quick Reference — Part 1

Concept / Method	What it does
JpaRepository<T, ID>	Top-level repo – inherits all CRUD + Paging + JPA extras
getReferenceById(id)	Returns a lazy proxy; DB hit deferred until field access
Example.of(entity)	Build WHERE clause from non-null fields of an entity object
repo.findAll(example)	Execute a Query by Example search
deleteAllById(ids)	N individual DELETE statements – one per id
deleteAllByIdInBatch(ids)	Single DELETE IN (...) – one DB round-trip, faster

Derived Query Methods

1. The Core Idea

Derived Query Methods are one of the most powerful features of Spring Data JPA. You write **zero SQL** — Spring reads the repository method name and generates the correct query automatically at runtime.

The naming formula is:

findBy + [FieldName] + [Keyword]

For example, Spring sees **findByPriceGreaterThan** and translates it to:

```
SELECT * FROM product WHERE price > ?
```

■ *No implementation needed — ever. Spring generates a proxy at startup that handles everything.*

2. Repository — IProductRepo.java

```
package com.telusko.repo;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.telusko.entity.Product;

@Repository
public interface IProductRepo extends JpaRepository<Product, Integer>
{
    // Equality
    List<Product> findByPrice(Double price);
    List<Product> findByPriceIs(Double price);
    List<Product> findByPriceEquals(Double price);

    // Comparison
    List<Product> findByPriceGreaterThan(Double price);
    List<Product> findByPriceLessThan(Double price);

    // Range
    List<Product> findByPriceBetween(Double minPrice, Double maxPrice);

    // Single field string match
    List<Product> findByCategoryEquals(String category);
}
```

```
// Combined: IN + BETWEEN
List<Product> findByCategoryInAndPriceBetween(
    List<String> categories, Double minPrice, Double maxPrice);
}
```

3. Service Interface — IProductService.java

```
package com.telusko.service;

import java.util.List;
import com.telusko.entity.Product;

public interface IProductService
{
    List<Product> searchByPrice(Double price);
    List<Product> searchByPriceGreaterThan(Double price);
    List<Product> searchByPriceLessThan(Double price);
    List<Product> searchByPriceBetween(Double minPrice, Double maxPrice);
    List<Product> searchByCategoryEquals(String category);
    List<Product> searchByCategoryInAndPriceBetween(
        List<String> categories, Double minPrice, Double maxPrice);
}
```

4. Service Implementation — ProductService.java

```
@Service
public class ProductService implements IProductService
{
    @Autowired
    private IProductRepo repo;

    @Override
    public List<Product> searchByPrice(Double price)
    {
        // findByPrice, findByPriceIs, findByPriceEquals all do the same thing
        return repo.findByPriceEquals(price);
    }

    @Override
    public List<Product> searchByPriceGreaterThan(Double price)
    {
        return repo.findByPriceGreaterThan(price);
    }

    @Override
```



```

public List<Product> searchByPriceLessThan(Double price)
{
    return repo.findByPriceLessThan(price);
}

@Override
public List<Product> searchByPriceBetween(Double minPrice, Double maxPrice)
{
    return repo.findByPriceBetween(minPrice, maxPrice);
}

@Override
public List<Product> searchByCategoryEquals(String category)
{
    return repo.findByCategoryEquals(category);
}

@Override
public List<Product> searchByCategoryInAndPriceBetween(
    List<String> categories, Double minPrice, Double maxPrice)
{
    return repo.findByCategoryInAndPriceBetween(categories, minPrice, maxPrice);
}
}

```

5. Key Concepts Explained

5.1 Equality — Three Ways to Write the Same Query

```

List<Product> findByPrice(Double price);      // implicit equals
List<Product> findByPriceIs(Double price);    // readable alias
List<Product> findByPriceEquals(Double price); // explicit equals

```

All three generate identical SQL:

```
SELECT * FROM product WHERE price = ?
```

Is and **Equals** are purely readability aliases. Use whichever reads most naturally in context. The service uses *findByPriceEquals* to be explicit.

5.2 Comparison Keywords

```

List<Product> findByPriceGreaterThan(Double price); // WHERE price > ?
List<Product> findByPriceLessThan(Double price);    // WHERE price < ?

```

Method Name	SQL Generated
-------------	---------------

<code>findByPriceGreaterThan(550.0)</code>	<code>WHERE price > 550.0</code>
<code>findByPriceLessThan(550.0)</code>	<code>WHERE price < 550.0</code>
<code>findByPriceGreaterThanEqual(550.0)</code>	<code>WHERE price >= 550.0</code>
<code>findByPriceLessThanEqual(550.0)</code>	<code>WHERE price <= 550.0</code>

5.3 Range — Between

```
List<Product> findByPriceBetween(Double minPrice, Double maxPrice);
```

```
// Generated SQL:
```

```
SELECT * FROM product WHERE price BETWEEN 450.0 AND 5001.0
```

Takes **two parameters** — min and max. Both ends are **inclusive**. The parameter names don't matter; only their position does.

5.4 Single Field String Match

```
List<Product> findByCategoryEquals(String category);
```

```
// Generated SQL:
```

```
SELECT * FROM product WHERE category = 'Fashion'
```

5.5 Combining Conditions — In + And + Between

This is the most powerful method in this code. Breaking the name down:

```
findBy  Category  In  And  Price  Between
      |         |   |   |         |
      field    keyword join field  keyword
```

```
List<Product> findByCategoryInAndPriceBetween(
    List<String> categories,
    Double minPrice,
    Double maxPrice
);
```

```
// Generated SQL:
```

```
SELECT * FROM product
WHERE category IN ('Fashion', 'Electronic')
AND price BETWEEN 450.0 AND 6000.0
```

In takes a *List<String>* and matches any value in that list — equivalent to multiple OR conditions. Combined with **And + Between**, you get a powerful multi-condition filter in a single method name with zero SQL.

6. Main Application

```
@SpringBootApplication
public class Datajpa1crudApplication {

    public static void main(String[] args) {
        ConfigurableApplicationContext container =
            SpringApplication.run(Datajpa1crudApplication.class, args);
        IProductService service = container.getBean(IProductService.class);

        // Search by exact price
        // service.searchByPrice(5000.0).forEach(p -> System.out.println(p));

        // Search price > 550
        // service.searchByPriceGreaterThan(550.0).forEach(p -> System.out.println(p));

        // Search price < 550
        // service.searchByPriceLessThan(550.0).forEach(p -> System.out.println(p));

        // Search price between 450 and 5001
        // service.searchByPriceBetween(450.0, 5001.0).forEach(p -> System.out.println(p));

        // Search by category
        // service.searchByCategoryEquals("Fashion").forEach(p -> System.out.println(p));

        // Combined: category IN list AND price BETWEEN range (active)
        List<String> categories = Arrays.asList("Fashion", "Electronic");
        service.searchByCategoryInAndPriceBetween(categories, 450.0, 6000.0)
            .forEach(p -> System.out.println(p));
    }
}
```

7. Full Keyword Reference

Keyword	Example Method	SQL Equivalent
(none) / Is / Equals	findByPrice	WHERE price = ?
GreaterThan	findByPriceGreaterThan	WHERE price > ?
LessThan	findByPriceLessThan	WHERE price < ?
GreaterThanEqual	findByPriceGreaterThanEqual	WHERE price >= ?
LessThanEqual	findByPriceLessThanEqual	WHERE price <= ?
Between	findByPriceBetween	WHERE price BETWEEN ? AND ?

In	findByCategoryIn	WHERE category IN (...)
And	findByCategoryAndPrice	WHERE category = ? AND price = ?
Or	findByCategoryOrPrice	WHERE category = ? OR price = ?
Like	findByProductNameLike	WHERE product_name LIKE ?
Containing	findByProductNameContaining	WHERE product_name LIKE '%?%'
OrderBy...Asc	findByCategoryOrderByPriceAsc	WHERE category = ? ORDER BY price ASC
OrderBy...Desc	findByCategoryOrderByPriceDesc	WHERE category = ? ORDER BY price DESC
Not	findByPriceNot	WHERE price != ?
IsNull	findByPriceIsNull	WHERE price IS NULL
IsNotNull	findByPriceIsNotNull	WHERE price IS NOT NULL

8. Quick Reference Summary — Part 2

Concept	Purpose
findBy + Field + Keyword	Core naming formula for all derived queries
findByPrice / Is / Equals	Exact match – all three generate WHERE price = ?
GreaterThan / LessThan	Comparison operators > and <
Between(min, max)	Inclusive range – WHERE price BETWEEN ? AND ?
In(list)	Match any value in a list – WHERE field IN (...)
And / Or	Combine multiple conditions in one method name
findByCategoryInAndPriceBetween	Category from a list AND price in a range